

FACULTY OF COMPUTING AND INFORMATION TECHNOLOGY

Bachelor of Software Engineering (Honours)

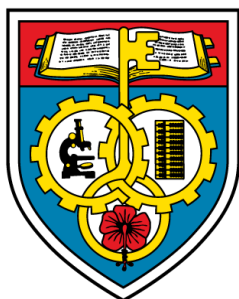
Programme: RSW (Group: 1)

99 Speed Market Application System

BACS2083 FORMAL METHODS FOR SOFTWARE ENGINEERING

Name (Block Letters)	Registration No.	Signature	Marks
1.Chan Jing Xing	2301830	<i>chan</i>	
2.Dylan Kong Jun Hong	2303775	<i>dylan</i>	
3.Tan Siew Wei	2302526	<i>tan</i>	
4.Yap Wing Hong	2301828	<i>yap</i>	
5.Ng Jien Hao	2214349	<i>hao</i>	

Tutor's Name: Ms. Azurawati Binti Ismail



FACULTY OF COMPUTING AND INFORMATION TECHNOLOGY

Plagiarism Statement and Guideline for Late Submission of Coursework

Read, complete, and sign this statement to be submitted with the written report.

We confirm that the submitted work are all our own work and are in our own words.

Name (Block Letters)	Registration No.	Signature	Date	Profile Picture
1.Chan Jing Xing	2301830	<i>chan</i>	30/12/2024	
2.Dylan Kong Jun Hong	2303775	<i>dylan</i>	30/12/2024	
3.Tan Siew Wei	2302526	<i>tan</i>	30/12/2024	
4.Yap Wing Hong	2301828	<i>yap</i>	30/12/2024	

5.Ng Jien Hao	2214349	<i>hao</i>	30/12/2024	
---------------	---------	-------------------	------------	---

Table Menu

1.0	Introduction	6
2.0	Requirement List	8
2.1	Account Management	8
2.2	Order	8
2.3	Payment	9
2.4	Membership	9
2.5	Customer Service & Support	10
3.0	Data Abstraction	11
3.1	Basic type	11
3.2	Free type	12
3.3	Axiomatic Definition	13
4.0	Z Schema	14
4.1	Account Management (Chan Jing Xing)	14
4.1.1	State Space Schema	14
4.1.2	Initial State Space Scheme	15
4.1.3	Operation Schema	15
4.1.4	Error Schema Scenario	19
4.1.5	Complete Schema	22
4.2	Order (Yap Wing Hong)	23
4.2.1	State Space Schema	23
4.2.2	Initial State Space Scheme	24
4.2.3	Operation Schema	25
4.2.4	Error Schema Scenario	28
4.2.5	Complete Schema	32
4.3	Payment (Ng Jien Hao)	33
4.3.1	State Space Schema	33
4.3.2	Initial State Space Scheme	33
4.3.3	Operation Schema	34
4.3.4	Error Schema Scenario	37
4.3.5	Complete Schema	39
4.4	Membership (Dylan Kong Jun Hong)	40
4.4.1	State Space Schema	40
4.4.2	Initial State Space Scheme	41
4.4.3	Operation Schema	42
4.4.4	Error Scenario Schema	47

4.4.5 Complete Schema	48
4.5 Customer Service & Support (Tan Siew Wei)	50
4.5.1 State Space Schema	50
4.5.2 Initial State Space Scheme	51
4.5.3 Operation Schema	52
4.5.4 Error Schema Scenario	58
4.5.5 Complete Schema	61
5.0 Self-Reflection	62
6.0 Reference and Appendices	64

1.0 Introduction

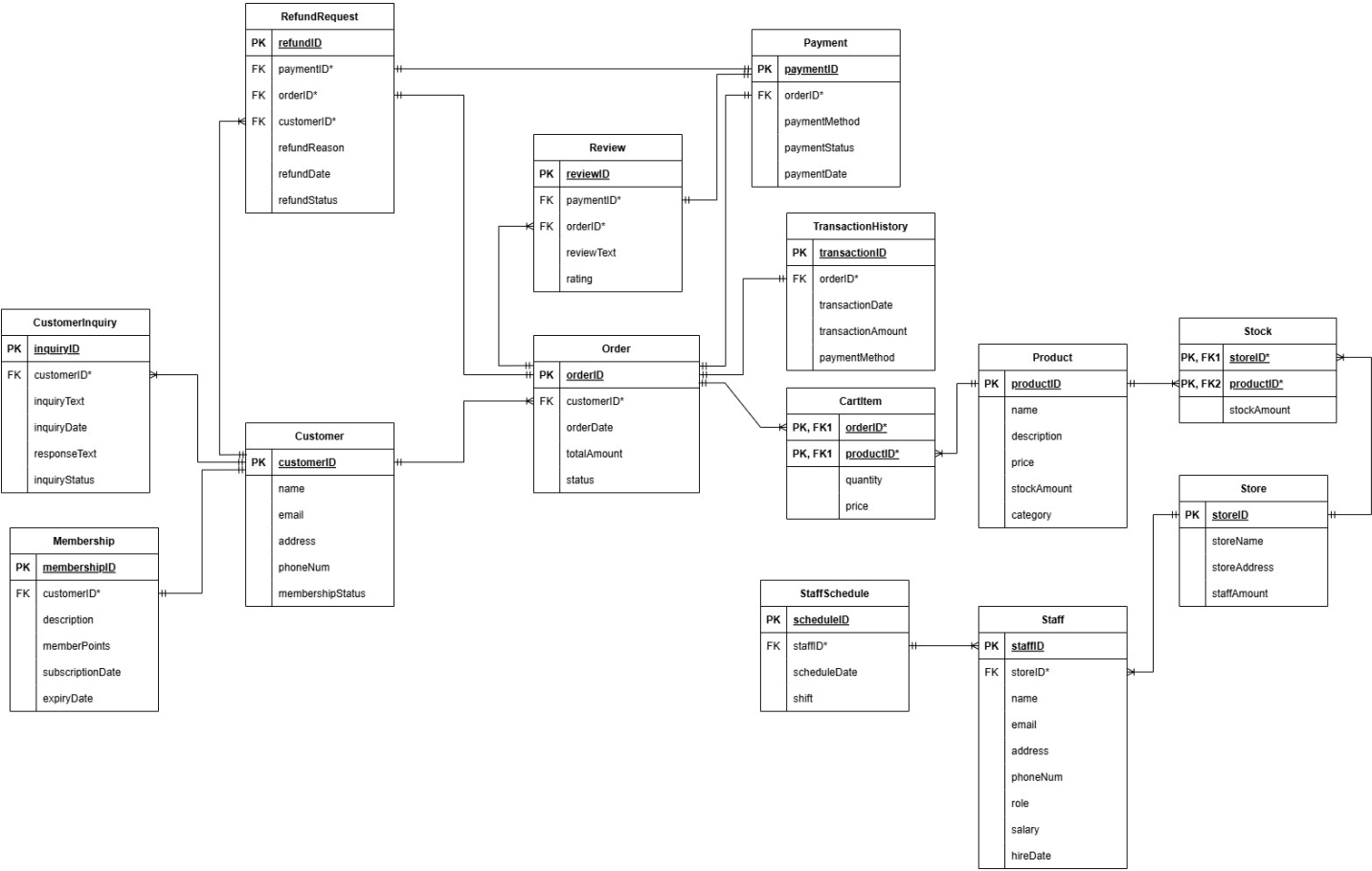
We are creating a point-of-sales mobile application for 99 Speedmart to align with the current trends in convenience stores and modern-day shopping. This application aims to allow customers to shop online, make operations more convenient for staff, and offer a personalized shopping experience for customers.

Several challenges faced by customers and staff are addressed through this mobile application. For instance, customers who wish to shop at their convenience, even outside of store hours, can now purchase items anytime. Those who find it inconvenient to visit a physical store can order items from anywhere using the application. Additionally, the app's product catalog enables customers to search for or filter products, saving time spent searching in-store.

Promotions and deals often go unnoticed by customers, but the application solves this by providing consistent notifications about ongoing offers. To address the issue of misplaced physical receipts, the app includes a transaction history feature, allowing customers to easily track their purchases and process refunds if necessary. Customers who wish to share their experiences with products can also write reviews directly within the app, enhancing engagement.

On the administrative side, the application centralizes staff management, making it easier for admins to edit schedules, assign roles, and notify staff. Overall, this mobile application provides a comprehensive solution to improve customer convenience, streamline operations, and enhance the shopping experience at 99 Speedmart.

ERD Diagram



2.0 Requirement List

2.1 Account Management

Customer :

- 2.1.1.1 The system shall allow the customer to register an account.
- 2.1.1.2 The system shall allow the customer to login to the system.
- 2.1.1.3 The system shall allow the customer to logout from the system.
- 2.1.1.4 The system shall allow the customer to recover the password for their account.
- 2.1.1.5 The system shall allow the customer to update their account details.
- 2.1.1.6 The system shall allow the customer to delete their account.
- 2.1.1.7 The system shall allow the customer to view transaction history.
- 2.1.1.8 The system shall allow the customer to subscribe to membership.
- 2.1.1.9 The system shall allow the customer to view their membership points.
- 2.1.1.10 The system shall allow the customer to redeem membership points.
- 2.1.1.11 The system shall allow the customer to access exclusive member deals.

Staff :

- 2.1.2.1 The system shall allow the staff to register, login, logout, and recover password.
- 2.1.2.2 The system shall allow the staff to update their account details.

2.2 Order

Customer :

- 2.2.1 The system shall allow the customer to view a detailed view of their order.
- 2.2.2 The system shall allow the customer to remove items from their order.
- 2.2.3 The system shall allow the customer to add items to their order.
- 2.2.4 The system shall allow the customer to adjust the quantity of items in the order.
- 2.2.5 The system shall allow the customer to cancel their order and remove all items from the order.
- 2.2.6 The system shall allow the customer to track their order's current status and location.
- 2.2.7 The system shall allow the customer to confirm their order and proceed to payment.
- 2.2.8 The system shall calculate the total price of the items in the cart.
- 2.2.9 The system shall display the total price and items within the order.

2.3 Payment

Customer :

2.3.1 The system shall allow the user to enter payment methods and details.

2.3.2 The system shall verify the payment details.

2.3.3 The system shall notify the user of the payment confirmation.

2.3.5 The system shall allow users to authenticate payment with security or biometric keys.

2.3.6 The system shall block authentication after 3 times failure.

2.3.7 The system shall generate receipt automatically

2.3.8 The system shall enable the printing receipt feature.

2.3.9 The system shall allow the user to request for a refund.

2.3.10 The system shall cancel payment after request for refund.

2.4 Membership

Customer :

2.4.1.1 The system shall allow customers to create a new membership account.

2.4.1.2 The system shall allow customers to view their membership details.

2.4.1.3 The system shall allow customers to check their membership status.

2.4.1.4 The system shall allow customers to update their membership account details.

2.4.1.5 The system shall allow customers to delete their membership account.

2.4.1.6 The system shall allow customers to add points to their account.

2.4.1.7 The system shall allow customers to check their available points balance.

2.4.1.8 The system shall allow customers to redeem points for rewards.

2.4.1.9 The system shall allow customers to receive a redemption confirmation after redeeming points.

2.4.1.10 The system shall allow customers to view exclusive deals available to redeem.

Staff :

2.4.2.1 The system shall allow staff to view and maintain membership account details.

2.4.2.2 The system shall allow staff to create new exclusive deals.

2.4.2.3 The system shall allow staff to search and view exclusive deals.

2.4.2.4 The system shall allow staff to update details of existing exclusive deals.

2.4.2.5 The system shall allow staff to delete exclusive deals.

2.4.2.6 The system shall allow staff to set categories for exclusive deals.

2.4.2.7 The system shall allow staff to update the points ledger by adding points to customer accounts.

2.4.2.8 The system shall notify customers when points are added to their account.

2.5 Customer Service & Support

Customer :

- 2.5.1.1 The system shall allow customers to submit inquiries regarding problems or support.
- 2.5.1.2 The system shall allow customers to update their filed inquiries through the system.
- 2.5.1.3 The system shall allow customers to attach supporting documents to their inquiries.
- 2.5.1.4 The system shall allow customers to search for previously submitted inquiries.
- 2.5.1.5 The system shall allow customers to initiate live chat sessions with staff for real-time assistance.
- 2.5.1.6 The system shall allow customers to send messages and attach files during a live chat session.
- 2.5.1.7 The system shall allow customers to view the history of their live chat sessions.
- 2.5.1.8 The system shall allow customers to search the FAQ database for information.
- 2.5.1.9 The system shall display FAQs for customers to read.
- 2.5.1.10 The system shall allow customers to submit feedback about their experience or services received.
- 2.5.1.11 The system shall allow customers to attach additional comments to their feedback.
- 2.5.1.12 The system shall allow customers to view details of their refund requests.
- 2.5.1.13 The system shall allow customers to check the status of their refund requests.

Staff :

- 2.5.2.1 The system shall allow staff to view and manage customer inquiries.
- 2.5.2.2 The system shall allow staff to send replies to customer inquiries.
- 2.5.2.3 The system shall allow staff to attach files to their responses.
- 2.5.2.4 The system shall allow staff to escalate inquiries to a specialist for further resolution.
- 2.5.2.5 The system shall allow staff to engage in live chat sessions with customers.
- 2.5.2.6 The system shall allow staff to send messages and attach files during live chat sessions.
- 2.5.2.7 The system shall allow staff to validate the authenticity of refund requests submitted by customers.
- 2.5.2.8 The system shall allow staff to make adjustments or modifications to refund requests.
- 2.5.2.9 The system shall allow staff to approve and process refund requests.
- 2.5.2.10 The system shall allow staff to request feedback from customers after resolving issues.
- 2.5.2.11 The system shall allow staff to request additional documents from customers when necessary.
- 2.5.2.12 The system shall allow staff to record the history of customer inquiries and their resolutions for future reference.
- 2.5.2.13 The system shall allow staff to track the status of refund requests.

3.0 Data Abstraction

3.1 Basic type

1. [CUSTOMERID] - the set of all customer id in the system.
2. [STAFFID] - the set of all staff id in the system.
3. [MEMBERSHIPID] - the set of all membership id in the system.
4. [PRODUCTID] - the set of all product id in the system.
5. [ORDERID] - the set of all order id in the system.
6. [PAYMENTID] - the set of all payment id in the system.
7. [TRANSACTIONID] - the set of all transaction id in the system.
8. [SCHEDULEID] - the set of all schedule id in the system.
9. [STOREID] - the set of all store id in the system.
10. [NAME] - the set of all names available in the system.
11. [ADDRESS] - the set of all addresses in the system.
12. [EMAIL] - the set of all email addresses in the system.
13. [DATE] - the set of all dates in the system.
14. [TIME] - the set of all times in the system.
15. [TEXT] - the set of inquiry, response, refund request, review text available in the system.
16. [DESCRIPTION] - the set of all descriptions in the system.
17. [PHONENUMBER] - the set of all phone numbers in the system.
18. [MEMBERSHIPLEVE] - the set of all membership levels in the system
19. [MEMBERSHIPSTATUS] - the set of all membership status in the system.
20. [POINT] - the set of all membership points in the system.
21. [ENQUIRYSTATUS] - the set of all enquiry status available in the system.
22. [ENQUIRYPRIORITY] - the set of all enquiry priority available in the system.
23. [ENQUIRYTEXT] - the set of all enquiry text available in the system.
24. [FEEDBACKTEXT] - the set of all feedback text available in the system.
25. [BOOLEAN] - the set of all true or false available in the system.
26. [ORDERREFUNDSTATUS] - the set of all order refund request status available in the system.
27. [ORDERREFUNDTEXT] - the set of all order refund request text available in the system.
28. [FOLLOWUPDATE] - the set of all followed update enquiry available in the system.
29. [STAFFRESPONSE] - the set of all staff responses available in the system.

3.2 Free type

1. PAYMENTMETHOD ::= tng | online banking | credit card | debit card | cash on delivery
2. MEMBERSHIPSTATUS ::= member | non-member
3. ORDERSTATUS ::= pending | confirmed | cancelled
4. PAYMENTSTATUS ::= pending | successful | cancelled
5. RATING ::= 1 | 2 | 3 | 4 | 5
6. ROLE ::= admin | manager | supervisor | cashier | cleaner | clerk | storekeeper | purchaser
7. ENQUIRYSTATUS ::= pending | resolved | reviewed
8. ENQUIRYPRIORITY ::= none | low | high
9. BOOLEAN ::= true | false
10. ORDERREFUNDSTATUS ::= requested | approved | rejected
11. DELIVERYSTATUS ::= pending | processing | shipped | delivered | canceled
12. ORDERERRORMESSAGE ::= order_not_exist | cart_limit_exceeded | insufficient_stock |
order_cancelled | order_already_cancelled | order_already_confirmed |
insufficient_stock_when_confirm_order | success

3.3 Axiomatic Definition

1. Cart Limit

$MaxCartLimit : \mathbb{N}$
$maxCartLimit : 30$

2. Order Minimum

$minOrderLimit : \mathbb{N}$
$mimOrderLimit : 15$

3. Inquiry Month Limit

$inquiryMonthLimit : \mathbb{N}$
$inquiryMonthLimit: 1$

4. Character Limit

$maxCharacterLimit : \mathbb{P}$
$maxCharacterLimit : 30$

5. Stock Limit

$maxStockLimit : \mathbb{N}$
$maxStockLimit : 100$

6. Spending Limit

$maxSpendLimit : \mathbb{N}$
$maxSpendLimit : 1000$

7. Membership Subscription Duration (1 Month)

$maxLimitSubscription : \mathbb{N}$
$maxLimitSubscription : 2505600 \text{ s}$

4.0 Z Schema

4.1 Account Management (Chan Jing Xing)

4.1.1 State Space Schema

Customer

$customerId : \mathbb{P} \text{ CUSTOMERID}$

$name : \mathbb{P} (\text{CUSTOMERID} \times \text{NAME})$

$email : \mathbb{P} (\text{CUSTOMERID} \times \text{EMAIL})$

$phoneNum : \mathbb{P} (\text{CUSTOMERID} \times \text{PHONENUM})$

$membershipStatus : \mathbb{P} (\text{CUSTOMERID} \times \text{MEMBERSHIPSTATUS})$

$\text{dom } membershipStatus \subseteq \text{dom } phoneNum \subseteq \text{dom } email \subseteq \text{dom } name \subseteq customerId$

TransactionHistory

$transactionId : \mathbb{P} \text{ TRANSACTIONID}$

$orderId : \mathbb{P} (\text{TRANSACTIONID} \times \text{ORDERID})$

$transactionDate : \mathbb{P} (\text{TRANSACTIONID} \times \text{TRANSACTIONDATE})$

$transactionAmount : \mathbb{P} (\text{TRANSACTIONID} \times \text{TRANSACTIONAMOUNT})$

$paymentMethod : \mathbb{P} (\text{TRANSACTIONID} \times \text{PAYMENTMETHOD})$

$\text{dom } paymentMethod \subseteq \text{dom } transactionAmount \subseteq \text{dom } transactionDate \subseteq \text{dom } orderId \subseteq transactionId$

Staff

$staffId : \mathbb{P} \text{ STAFFID}$

$name : \mathbb{P} (\text{STAFFID} \times \text{NAME})$

$email : \mathbb{P} (\text{STAFFID} \times \text{EMAIL})$

$phoneNum : \mathbb{P} (\text{STAFFID} \times \text{PHONENUM})$

$role : \mathbb{P} (\text{STAFFID} \times \text{ROLE})$

$salary : \mathbb{P} (\text{STAFFID} \times \text{SALARY})$

$hireDate : \mathbb{P} (\text{STAFFID} \times \text{HIREDATE})$

$\text{dom } hireDate \subseteq \text{dom } salary \subseteq \text{dom } role \subseteq \text{dom } phoneNum \subseteq \text{dom } email \subseteq \text{dom } name \subseteq staffId$

4.1.2 Initial State Space Scheme

<i>InitCustomer</i>
<i>Customer</i>
$customerId = \emptyset$

<i>InitStaff</i>
<i>Staff</i>
$staffId = \emptyset$

4.1.3 Operation Schema

4.1.3.1 Operation 1: CRUD

<i>AddCustomer</i>
Δ <i>Customer</i> $cId? : CUSTOMERID$ $name? : NAME$ $email? : EMAIL$ $pNum?: \mathbb{N}_1$
$cId? \notin customerId$ $phoneNum? \leq 11$ $customerId' = customerId \cup \{cId?\}$ $name' = name \cup \{cId? \mapsto name?\}$ $email' = email \cup \{cId? \mapsto email?\}$ $phoneNum' = phoneNum \cup \{cId? \mapsto pNum?\}$

<i>UpdateCustomer</i>
Δ <i>Customer</i> $cId? : CUSTOMERID$ $name? : NAME$ $email? : EMAIL$ $pNum?: \mathbb{N}_1$
$cId? \in customerId$ $customerId' = customerId \oplus \{cId?\}$ $name' = name \oplus \{cId? \mapsto name?\}$ $email' = email \oplus \{cId? \mapsto email?\}$ $phoneNum' = phoneNum \oplus \{cId? \mapsto pNum?\}$ $customerId' = customerId$

RemoveCustomer

Δ Customer

$cId? : CUSTOMERID$

$name? : NAME$

$email? : EMAIL$

$pNum?: \mathbb{N}_1$

$cId? \in customerId$

$customerId' = customerId \setminus \{cId?\}$

$name' = name \setminus \{cId? \mapsto name?\}$

$email' = email \setminus \{cId? \mapsto email?\}$

$phoneNum' = phoneNum \setminus \{cId? \mapsto pNum?\}$

AddStaff

Δ Staff

$sId? : STAFFID$

$name? : NAME$

$email? : EMAIL$

$pNum? : \mathbb{N}_1$

$role? : ROLE$

$hireDate? : HIREDATE$

$sId? \notin staffId$

$phoneNum? \leq 11$

$staffId' = staffId \cup \{sId?\}$

$name' = name \cup \{sId? \mapsto name?\}$

$email' = email \cup \{sId? \mapsto email?\}$

$phoneNum' = phoneNum \cup \{sId? \mapsto pNum?\}$

$role' = role \cup \{sId? \mapsto role?\}$

$hireDate' = hireDate \cup \{sId? \mapsto hireDate?\}$

UpdateStaff

Δ Staff

$sId? : STAFFID$

$name? : NAME$

$email? : EMAIL$

$pNum? : \mathbb{N}_1$

$role? : ROLE$

$hireDate? : HIREDATE$

$sId? \in staffId$

$staffId' = staffId$

$name' = name \oplus \{sId? \mapsto name?\}$

$email' = email \oplus \{sId? \mapsto email?\}$

$phoneNum' = phoneNum \oplus \{sId? \mapsto pNum?\}$

$role' = role \oplus \{sId? \mapsto role?\}$

$hireDate' = hireDate \oplus \{sId? \mapsto hireDate?\}$

RemoveStaff

Δ Staff

$sId? : STAFFID$

$name? : NAME$

$email? : EMAIL$

$pNum? : \mathbb{N}_1$

$role? : ROLE$

$hireDate? : HIREDATE$

$sId? \in staffId$

$staffId' = staffId \setminus \{sId?\}$

$name' = name \setminus \{sId? \mapsto name?\}$

$email' = email \setminus \{sId? \mapsto email?\}$

$phoneNum' = phoneNum \setminus \{sId? \mapsto pNum?\}$

$role' = role \setminus \{sId? \mapsto role?\}$

$hireDate' = hireDate \setminus \{sId? \mapsto hireDate?\}$

4.1.3.2 Operation 2: Recover Password

RecoverPassword

$\exists$ *Customer*

email? : *EMAIL*

success! : *BOOLEAN*

email? $\in \text{ran}(\text{email})$

success! = *true*

4.1.3.3 Operation 3: Subscribe Membership

SubscribeMembership

Δ *Customer*

cId? : *CUSTOMERID*

success! : *BOOLEAN*

cId? $\in \text{customerId}$

cId? $\notin \text{dom}(\text{membershipStatus})$

$\text{membershipStatus}' = \text{membershipStatus} \cup \{cId? \mapsto \text{TRUE}\}$

success! = *TRUE*

4.1.4 Error Schema Scenario

Schema Name	Success Pre-condition	Failure Pre-condition	Remark for Failure
AddCustomerError	$cId? \notin \text{customerId} \wedge pNum? \leq 11$	$cId? \in \text{customerId} \vee pNum? > 11$	"Customer ID already exists or phone number is invalid."
UpdateCustomerError	$cId? \in \text{customerId}$	$cId? \notin \text{customerId}$	"Customer ID does not exist."
RemoveCustomerError	$cId? \in \text{customerId}$	$cId? \notin \text{customerId}$	"Customer ID does not exist."
AddStaffError	$sId? \notin \text{staffId} \wedge pNum? \leq 11$	$sId? \in \text{staffId} \vee pNum? > 11$	"Staff ID already exists or phone number is invalid."
UpdateStaffError	$sId? \in \text{staffId}$	$sId? \notin \text{staffId}$	"Staff ID does not exist."
RemoveStaffError	$sId? \in \text{staffId}$	$sId? \notin \text{staffId}$	"Staff ID does not exist."
RecoverPasswordError	$\text{email}? \in \text{ran}(\text{email})$	$\text{email}? \notin \text{ran}(\text{email})$	"Email does not exist."
SubscribeMembershipError	$cId? \in \text{customerId} \wedge cId? \notin \text{dom}(\text{membershipStatus})$	$cId? \notin \text{customerId} \vee cId? \in \text{dom}(\text{membershipStatus})$	"Customer ID does not exist or is already subscribed."

MESSAGE ::= customer_exist | phoneNumber_invalid | customer_not_exist | staff_exist | staff_not_exist | email_not_exist | membership_exist | success

Success

rep! : MESSAGE

rep! = success

AddCustomerError

$\exists$ Customer

cId? : CUSTOMERID

pNum? : $\mathbb{N}_1$

error! : STRING

$(cId? \in \text{customerId} \wedge \text{error!} = \text{"Customer ID already exist."})$

$\vee$

$(pNum? > 11 \wedge \text{error!} = \text{"Phone number is invalid."})$

UpdateCustomerError

$\exists$ Customer

$cId? : CUSTOMERID$

$error! : STRING$

$(cId? \notin customerId \wedge error! = \text{"Customer ID does not exist."})$

RemoveCustomerError

$\exists$ Customer

$cId? : CUSTOMERID$

$error! : STRING$

$(cId? \notin customerId \wedge error! = \text{"Customer ID does not exist."})$

AddStaffError

$\exists$ Staff

$sId? : STAFFID$

$pNum? : \mathbb{N}$

$error! : STRING$

$(sId? \in staffId \wedge error! = \text{"Staff ID already exists."})$

$\vee$

$(pNum? > 11 \wedge error! = \text{"Phone number is invalid."})$

UpdateStaffError

$\exists$ Staff

$sId? : STAFFID$

$error! : STRING$

$(sId? \notin staffId \wedge error! = \text{"Staff ID does not exist."})$

RemoveStaffError

$\exists$ Staff

$sId? : STAFFID$

$error! : STRING$

$(sId? \notin staffId \wedge error! = \text{"Staff ID does not exist."})$

RecoverPasswordError

$\exists$ *Customer*

email? : *EMAIL*

error! : *STRING*

$(email? \notin \text{ran}(\text{email}) \wedge error! = \text{"Email does not exist."})$

SubscribeMembershipError

$\exists$ *Customer*

cId? : *CUSTOMERID*

error! : *STRING*

$(cId? \notin \text{customerId} \wedge error! = \text{"Customer ID does not exist."})$

$\vee$

$(cId? \in \text{dom}(\text{membershipStatus}) \wedge error! = \text{"Customer ID is already subscribed."})$

4.1.5 Complete Schema

1. $\text{AddCustomerComplete} \triangleq (\text{AddCustomer} \wedge \text{SuccessSchema}) \vee \text{AddCustomerError}$
2. $\text{UpdateCustomerComplete} \triangleq (\text{UpdateCustomer} \wedge \text{SuccessSchema}) \vee \text{UpdateCustomerError}$
3. $\text{RemoveCustomerComplete} \triangleq (\text{RemoveCustomer} \wedge \text{SuccessSchema}) \vee \text{RemoveCustomerError}$
4. $\text{AddStaffComplete} \triangleq (\text{AddStaff} \wedge \text{SuccessSchema}) \vee \text{AddStaffError}$
5. $\text{UpdateStaffComplete} \triangleq (\text{UpdateStaff} \wedge \text{SuccessSchema}) \vee \text{UpdateStaffError}$
6. $\text{RemoveStaffComplete} \triangleq (\text{RemoveStaff} \wedge \text{SuccessSchema}) \vee \text{RemoveStaffError}$
7. $\text{RecoverPasswordComplete} \triangleq (\text{RecoverPassword} \wedge \text{SuccessSchema}) \vee \text{RecoverPasswordError}$
8. $\text{SubscribeMembershipComplete} \triangleq (\text{SubscribeMembership} \wedge \text{SuccessSchema}) \vee \text{SubscribeMembershipError}$

4.2 Order (Yap Wing Hong)

4.2.1 State Space Schema

Order

$orderId$: $\mathbb{P} \text{ ORDERID}$
 $customerId$: $\text{ORDERID} \rightarrow \text{CUSTOMERID}$
 $orderDate$: $\text{ORDERID} \rightarrow \text{DATE}$
 $orderTime$: $\text{ORDERID} \rightarrow \text{TIME}$
 $orderItem$: $\mathbb{P} \text{ ORDERID} \leftrightarrow \text{PRODUCTID}$
 $totalPrice$: $\text{ORDERID} \rightarrow \mathbb{N}_1$
 $orderStatus$: $\text{ORDERID} \rightarrow \text{ORDERSTATUS}$
 $shippingAddress$: $\text{ORDERID} \rightarrow \text{ADDRESS}$
 $orderCurrentLocation$: $\text{ORDERID} \rightarrow \text{ADDRESS}$
 $orderDeliveryStatus$: $\text{ORDERID} \rightarrow \text{DELIVERYSTATUS}$
 $orderPaymentStatus$: $\text{ORDERID} \rightarrow \text{PAYMENTSTATUS}$
 $orderPaymentMethod$: $\text{ORDERID} \rightarrow \text{PAYMENTMETHOD}$
 $itemAmount$: $\text{ORDERID} \times \text{PRODUCTID} \rightarrow \mathbb{N}_1$
 $cartLimit$: $\mathbb{N}$
 $orderMin$: $\mathbb{N}$

$\text{dom } customerId = \text{dom } orderDate = \text{dom } totalAmount = \text{dom } orderItem = \text{dom } orderStatus =$
 $\text{dom } shippingAddress = \text{dom } orderDeliveryStatus = \text{dom } orderPaymentStatus =$
 $\text{dom } orderPaymentMethod = \text{dom } itemAmount = \text{orderId}$

$cartLimit = 30$

$orderMin = 1$

$\forall orderId: \text{ORDERID} \bullet (\#(orderItem(orderId)) \leq cartLimit) \wedge$

$(\#(orderItem(orderId)) \geq orderMin) \wedge$

$(totalPrice(orderId) = \Sigma \{ itemAmount(orderId, productId) * itemPrice(productId) \mid productId:$
 $\text{PRODUCTID} \bullet orderItem(orderId, productId) \})$

Product

$productId$: $\mathbb{P} \text{ PRODUCTID}$
 $productName$: $\text{PRODUCTID} \rightarrow \text{TEXT}$
 $productPrice$: $\text{PRODUCTID} \rightarrow \mathbb{N}_1$
 $productDescription$: $\text{PRODUCTID} \rightarrow \text{DESCRIPTION}$
 $productStock$: $\text{PRODUCTID} \rightarrow \mathbb{N}_1$

$\text{dom } productName = \text{dom } productPrice = \text{dom } productDescription = \text{productStock} = \text{productId}$

4.2.2 Initial State Space Scheme

<i>InitOrder</i>
<i>Order</i>
<i>orderId</i> = $\emptyset$ <i>customerId</i> = $\emptyset$ <i>orderDate</i> = $\emptyset$ <i>orderTime</i> = $\emptyset$ <i>orderItem</i> = $\emptyset$ <i>totalPrice</i> = $\emptyset$ <i>orderStatus</i> = “ <i>pending</i> ” <i>shippingAddress</i> = $\emptyset$ <i>orderCurrentLocation</i> = $\emptyset$ <i>orderDeliveryStatus</i> = “ <i>pending</i> ” <i>orderPaymentStatus</i> = “ <i>pending</i> ” <i>orderPaymentMethod</i> = “ <i>unselected</i> ” <i>itemAmount</i> = $\emptyset$
<i>InitProduct</i>
<i>Product</i>
<i>productId</i> = $\emptyset$ <i>productName</i> = $\emptyset$ <i>productPrice</i> = $\emptyset$ <i>productDescription</i> = $\emptyset$ <i>productStock</i> = $\emptyset$

4.2.3 Operation Schema

AddNewOrderItem

Δ *Order*

ordId? : *ORDERID*

custId? : *CUSTID*

ordItem? : *PRODUCTID*

itemAmount? : $\mathbb{N}_1$

ordId? $\in$ *orderId*

custId? = *customerId(ordId?)*

$\#(\text{orderItem}(\text{ordId?})) < \text{cartLimit}$

productStock(ordItem?) $\geq$ *itemAmount?*

$(\text{ordId?}, \text{ordItem?}) \in \text{orderItem} \Rightarrow$

$\text{itemAmount}' = \text{itemAmount} \oplus \{(\text{ordId?}, \text{ordItem?}) \mapsto (\text{itemAmount}(\text{ordId?}, \text{ordItem?}) + \text{itemAmount?})\}$

$(\text{ordId?}, \text{ordItem?}) \notin \text{orderItem} \Rightarrow$

$\text{orderItem}' = \text{orderItem} \cup \{(\text{ordId?}, \text{ordItem?})\}$

$\text{itemAmount}' = \text{itemAmount} \oplus \{(\text{ordId?}, \text{ordItem?}) \mapsto \text{itemAmount?}\}$

$\text{totalPrice}' = \text{totalPrice} \oplus \{(\text{ordId?}) \mapsto (\text{totalPrice}(\text{ordId?}) + (\text{itemAmount?} * \text{productPrice}(\text{ordItem?})))\}$

$\text{productStock}' = \text{productStock} \oplus \{(\text{ordItem?}) \mapsto (\text{productStock}(\text{ordItem?}) - \text{itemAmount?})\}$

ViewOrderDetails

⊖ Order

⊖ Product

$ordId? : ORDERID$

$orderDate! : DATE$

$orderTime! : TIME$

$orderItem! : \mathbb{P} PRODUCTID$

$totalPrice! : \mathbb{N}_1$

$orderStatus! : ORDERSTATUS$

$shippingAddress! : ADDRESS$

$orderPaymentStatus! : PAYMENTSTATUS$

$orderPaymentMethod! : PAYMENTMETHOD$

$itemAmount! : \mathbb{N}_1$

$productName! : TEXT$

$productPrice! : \mathbb{N}_1$

$ordId? \in orderId$

$orderDate! = orderDate(ordId?)$

$orderTime! = orderTime(ordId?)$

$orderItem! = orderItem(ordId?)$

$totalPrice! = totalPrice(ordId?)$

$orderStatus! = orderStatus(ordId?)$

$shippingAddress! = shippingAddress(ordId?)$

$orderPaymentStatus! = orderPaymentStatus(ordId?)$

$orderPaymentMethod! = orderPaymentMethod(ordId?)$

$itemAmount! = itemAmount(ordId?)$

$productName! = \{name: TEXT \mid p: PRODUCTID \bullet p \in orderItem! \wedge name = productName(p)\}$

$productPrice! = \{price: \mathbb{N}_1 \mid p: PRODUCTID \bullet p \in orderItem! \wedge price = productPrice(p)\}$

$orderDate' = orderDate$

$orderTime' = orderTime$

$orderItem' = orderItem$

$totalPrice' = totalPrice$

$orderStatus' = orderStatus$

$shippingAddress' = shippingAddress$

$orderPaymentStatus' = orderPaymentStatus$

$orderPaymentMethod' = orderPaymentMethod$

$itemAmount' = itemAmount$

$productName' = productName$

$productPrice' = productPrice$

UpdateOrderDetails

Δ Order

$ordId? : ORDERID$

$orderItem? : \mathbb{P} PRODUCTID$

$itemAmount? : \mathbb{N}$

$shippingAddress? : ADDRESS$

$orderPaymentMethod? : PAYMENTMETHOD$

$ordId? \in orderId$

$orderStatus(ordId?) \neq \text{"cancelled"}$

$\forall p: PRODUCTID \bullet$

$(p \in orderItem? \Rightarrow$

$(itemAmount?(p) = 0 \Rightarrow orderItem' = orderItem \setminus \{(ordId?, p)\}) \wedge$

$(itemAmount?(p) > 0 \Rightarrow itemAmount' = itemAmount \oplus \{(ordId?, p) \mapsto itemAmount?(p)\})$

$shippingAddress? \neq \emptyset \Rightarrow$

$shippingAddress' = shippingAddress \oplus \{(ordId?) \mapsto shippingAddress?\}$

$orderPaymentMethod? \neq \emptyset \Rightarrow$

$orderPaymentMethod' = orderPaymentMethod \oplus \{(ordId?) \mapsto orderPaymentMethod?\}$

$totalPrice' = totalPrice \oplus \{(ordId?) \mapsto$

$\{itemAmount'(ordId?, p) * productPrice(p) \mid p: PRODUCTID \bullet p \in dom(itemAmount'(\{ordId?\}))\}\}$

CancelOrder

Δ Order

$ordId? : ORDERID$

$ordId? \in orderId \wedge orderStatus(ordId?) \neq \text{"cancelled"}$

$orderStatus' = orderStatus \oplus \{(ordId?) \mapsto \text{"cancelled"}\}$

$orderPaymentStatus' = orderPaymentStatus \oplus \{(ordId?) \mapsto \text{"cancelled"}\}$

$orderItem' = orderItem \setminus \{(ordId?, p) \mid p: PRODUCTID \bullet p \in dom(itemAmount(\{ordId?\}))\}$

$itemAmount' = itemAmount \setminus \{(ordId?, p) \mid p: PRODUCTID$

$totalPrice' = totalPrice \oplus \{(ordId?) \mapsto 0\}$

$shippingAddress' = shippingAddress \setminus \{ordId?\}$

$orderPaymentMethod' = orderPaymentMethod \setminus \{ordId?\}$

$\forall p: PRODUCTID \bullet$

$p \in dom(itemAmount(\{ordId?\})) \Rightarrow$

$productStock' = productStock \oplus \{(p \mapsto productStock(p) + itemAmount(ordId?, p))\}$

TrackingOrder

$\exists$ Order

$ordId? : ORDERID$

$orderStatus! : ORDERSTATUS$

$orderDeliveryStatus! : DELIVERYSTATUS$

$orderCurrentLocation! : ADDRESS$

$orderShippingAddress! : ADDRESS$

$estimatedDelivery! : DATE$

$ordId? \in orderId$

$orderStatus! = orderStatus(ordId?)$

$orderPaymentStatus! = orderPaymentStatus(ordId?)$

$orderDeliveryStatus! = orderDeliveryStatus(ordId?)$

$orderCurrentLocation! = orderCurrentLocation(ordId?)$

$shippingAddress! = shippingAddress(ordId?)$

$estimatedDelivery! = estimatedDelivery(ordId?)$

ConfirmOrder

Δ Order

$ordId? : ORDERID$

$ordId? \in orderId \wedge orderStatus(ordId?) \neq \text{"confirmed"}$

$orderStatus' = orderStatus \oplus \{(ordId?, \text{"confirmed"})\}$

$orderPaymentStatus' = orderPaymentStatus \oplus \{(ordId?) \mapsto \text{"successful"}\}$

$orderDeliveryStatus' = orderDeliveryStatus \oplus \{(ordId?) \mapsto \text{"processing"}\}$

$\forall p: PRODUCTID \bullet$

$p \in orderItem \langle \{ordId?\} \rangle \Rightarrow$

$productStock' = productStock \oplus \{(p \mapsto productStock(p) - itemAmount(ordId?, p))\}$

$totalPrice' = totalPrice(ordId?)$

$shippingAddress' = shippingAddress(ordId?)$

$orderPaymentMethod' = orderPaymentMethod(ordId?)$

4.2.4 Error Schema Scenario

Schema Name	Success Pre-condition	Failure Pre-condition	Remark for Failure
AddNewOrderItem	$ordId? \in orderId$ $\#(orderItem(ordId?)) < cartLimit$	$ordId? \notin orderId$ $\#(orderItem(ordId?)) \geq cartLimit$	1. Order does not exist

	$\text{productStock}(\text{ordItem?}) \geq \text{itemAmount?}$	$\text{productStock}(\text{ordItem?}) < \text{itemAmount?}$	2. Cart limit exceeded 3. Insufficient stock
ViewOrderDetails	$\text{ordId?} \in \text{orderId}$	$\text{ordId?} \in \text{orderId}$	1. Order does not exist
UpdateOrderDetails	$\text{ordId?} \in \text{orderId}$ $\text{orderStatus}(\text{ordId?}) \neq \text{"cancelled"}$	$\text{ordId?} \notin \text{orderId}$ $\text{orderStatus}(\text{ordId?}) = \text{"cancelled"}$	1. Order does not exist 2. Order is cancelled
CancelOrder	$\text{ordId?} \in \text{orderId}$ $\text{orderStatus}(\text{ordId?}) \neq \text{"cancelled"}$	$\text{ordId?} \notin \text{orderId}$ $\text{orderStatus}(\text{ordId?}) = \text{"cancelled"}$	1. Order does not exist 2. Order is already cancelled
TrackingOrder	$\text{ordId?} \in \text{orderId}$	$\text{ordId?} \notin \text{orderId}$	1. Order does not exist

ORDERERRORMESSAGE ::= order_not_exist | cart_limit_exceeded | insufficient_stock | order_cancelled | order_already_cancelled | order_already_confirmed | insufficient_stock_when_confirm_order | success

<u>OrderSuccessSchema</u>
$\text{rep!} : \text{ORDERERRORMESSAGE}$
$\text{rep!} = \text{success}$

AddNewOrderItemError

$\exists$ Order

$ordId? : ORDERID$

$ordItem? : PRODUCTID$

$itemAmount? : \mathbb{N}$

$rep! : ORDERERRORMESSAGE$

$(ordId? \notin orderId \wedge rep! = order_not_exist)$

$\vee$

$(\#(orderItem(ordId?)) \geq cartLimit \wedge rep! = cart_limit_exceeded)$

$\vee$

$(productStock(ordItem?) < itemAmount? \wedge rep! = insufficient_stock)$

ViewOrderDetailsError

$\exists$ Order

$ordId? : ORDERID$

$rep! : ORDERERRORMESSAGE$

$ordId? \notin orderId \wedge rep! = order_not_exist$

UpdateOrderDetailsError

$\exists$ Order

$ordId? : ORDERID$

$orderStatus? : STATUS$

$rep! : ORDERERRORMESSAGE$

$(ordId? \notin orderId \wedge rep! = order_not_exist)$

$\vee$

$(orderStatus(ordId?) = "cancelled" \wedge rep! = order_cancelled)$

CancelOrderError

$\exists$ Order

$ordId? : ORDERID$

$orderStatus? : STATUS$

$rep! : ORDERERRORMESSAGE$

$(ordId? \notin orderId \wedge rep! = order_not_exist)$

$\vee$

$(orderStatus(ordId?) = "cancelled" \wedge rep! = order_already_cancelled)$

TrackingOrderError

$\exists$ Order

$ordId? : ORDERID$

$rep! : ORDERERRORMESSAGE$

$ordId? \notin orderId \wedge rep! = order_not_exist$

ConfirmOrderError

$\exists$ Order

ordId? : ORDERID

orderStatus? : STATUS

ordItem? : ITEM

itemAmount? : $\mathbb{N}_1$

rep! : ORDERERRORMESSAGE

(ordId? $\notin$ orderId $\wedge$ rep! = order_not_exist)

$\vee$

(orderStatus(ordId?) = "confirmed" $\wedge$ rep! = order_already_confirmed)

$\vee$

(productStock(ordItem?) < itemAmount? $\wedge$ rep! = insufficient_stock_when_confirm_order)

4.2.5 Complete Schema

- 1) $\text{AddNewOrderItemComplete} \triangleq (\text{AddNewOrderItem} \wedge \text{OrderSuccessSchema}) \vee \text{AddNewOrderItemError}$
- 2) $\text{ViewOrderDetailsComplete} \triangleq (\text{ViewOrderDetails} \wedge \text{OrderSuccessSchema}) \vee \text{ViewOrderDetailsError}$
- 3) $\text{UpdateOrderDetailsComplete} \triangleq (\text{UpdateOrderDetails} \wedge \text{OrderSuccessSchema}) \vee \text{UpdateOrderDetailsError}$
- 4) $\text{CancelOrderComplete} \triangleq (\text{CancelOrder} \wedge \text{OrderSuccessSchema}) \vee \text{CancelOrderError}$
- 5) $\text{TrackingOrderComplete} \triangleq (\text{TrackingOrder} \wedge \text{OrderSuccessSchema}) \vee \text{TrackingOrderError}$
- 6) $\text{ConfirmOrderComplete} \triangleq (\text{ConfirmOrder} \wedge \text{OrderSuccessSchema}) \vee \text{ConfirmOrderError}$

4.3 Payment (Ng Jien Hao)

4.3.1 State Space Schema

<i>Payment</i>
<i>paymentId</i> : $\mathbb{P}$ <i>PAYMENTID</i>
<i>associatedOrderId</i> : $\mathbb{P}$ (<i>PAYMENTID</i> $\rightarrow$ <i>ORDERID</i>)
<i>paymentMethod</i> : $\mathbb{P}$ (<i>PAYMENTID</i> $\rightarrow$ <i>PAYMENTMETHOD</i>)
<i>paymentStatus</i> : $\mathbb{P}$ (<i>PAYMENTID</i> $\rightarrow$ <i>PAYMENTSTATUS</i>)
<i>paymentDate</i> : $\mathbb{P}$ (<i>PAYMENTID</i> $\rightarrow$ <i>DATE</i>)
<i>dom paymentId</i> = <i>dom associatedOrderId</i> = <i>dom paymentMethod</i> = <i>dom paymentStatus</i> = <i>dom paymentDate</i>

4.3.2 Initial State Space Scheme

<i>InitPayment</i>
<i>Payment</i>
<i>paymentId</i> = $\emptyset$

4.3.3 Operation Schema

4.3.3.1 Operation 1 : CRUD

NewPayment

Δ *Payment*

paymentId? : *PAYMENTID*

associatedOrderId? : *ORDERID*

paymentMethod? : *PAYMENTMETHOD*

paymentDate? : *DATE*

paymentId? $\notin$ *paymentId*

associatedOrderId? $\notin$ *associatedOrderId*

paymentId' = *paymentId* $\cup$ {*paymentId?*}

orderId' = *orderId* $\cup$ {*paymentId?* $\mapsto$ *orderId?*}

paymentMethod' = *paymentMethod* $\cup$ {*paymentId?* $\mapsto$ *paymentMethod?*}

paymentStatus' = *paymentStatus* $\cup$ {*paymentId?* $\mapsto$ *pending*}

paymentDate' = *paymentDate* $\cup$ {*paymentId?* $\mapsto$ *paymentDate?*}

ViewPaymentDetails

Ξ *Payment*

paymentId? : *PAYMENTID*

associatedOrderId? : *ORDERID*

paymentMethod? : *PAYMENTMETHOD*

paymentStatus! : *PAYMENTSTATUS*

paymentDate? : *PAYMENTDATE*

paymentID? $\in$ *paymentID*

associatedOrderId! = *associatedOrderId*(*paymentId?*)

paymentMethod! = *paymentMethod*(*paymentId?*)

paymentStatus! = *paymentStatus*(*paymentId?*)

paymentDate! = *paymentDate*(*paymentId?*)

paymentID' = *paymentID*

associatedOrderId' = *associatedOrderId*

paymentMethod' = *paymentMethod*

paymentStatus' = *paymentStatus*

paymentDate' = *paymentDate*

UpdatePaymentMethod

Δ Payment

$paymentID? : PAYMENTID$

$paymentMethod? : PAYMENTMETHOD$

$paymentID? \in paymentID$

$paymentMethod \neq paymentMethod?$

$paymentStatus \neq successful$

$paymentStatus \neq cancelled$

$paymentMethod' = paymentMethod \oplus \{paymentID? \mapsto paymentMethod?\}$

$associatedOrderId' = associatedOrderId$

$paymentStatus' = paymentStatus$

$paymentDate' = paymentDate$

RemovePayment

Δ Payment

$paymentId? : PAYMENTID$

$paymentID? \in paymentID$

$paymentStatus \neq successful$

$paymentId' = paymentId \setminus \{paymentId?\}$

$associatedOrderId' = associatedOrderId \setminus \{paymentId?\}$

$paymentStatus' = paymentStatus \setminus \{paymentId?\}$

$paymentDate' = paymentDate \setminus \{paymentId?\}$

$paymentStatus = paymentStatus \setminus \{paymentId?\}$

4.3.3.1 Operation 2: Update Payment Status

UpdatePaymentStatus

Δ *Payment*

paymentId? : *PAYMENTID*

paymentStatus? = *PAYMENTSTATUS*

paymentID? $\in$ *paymentID*

paymentStatus $\neq$ *paymetStatus?*

paymentStatus $\neq$ *successful*

paymentStatus $\neq$ *cancelled*

paymentStatus' = *paymentStatus* $\oplus$ {*paymentID?* $\mapsto$ *PAYMENTSTATUS*}

paymentID' = *paymentID*

associatedOrderId' = *associatedOrderId*

paymentStatus' = *paymentStatus*

paymentDate' = *paymentDate*

4.3.3.3 Operation 3 : Retrieve Payment Details Using Order ID

RetrievePaymentDetailsUsingOrderId

$\exists$ *Payment*

associatedOrderId? = *ORDERID*

paymentId! : *PAYMENTID*

associatedOrderId! : *ORDERID*

paymentMethod! : *PAYMENTMETHOD*

paymentStatus! : *PAYMENTSTATUS*

paymentDate! : *PAYMENTDATE*

associatedOrderId? $\in$ *ran* (*associatedOrderId* $\triangleright$ *associatedOrderId?*)

paymentId! = *dom* (*associatedOrderId* $\triangleright$ *associatedOrderId?*)

paymentMethod! = *paymentMethod*(*paymentId?*)

paymentStatus! = *paymentStatus*(*paymentId?*)

paymentDate! = *paymentDate*(*paymentId?*)

paymentID' = *paymentID*

associatedOrderId' = *associatedOrderId*

paymentMethod' = *paymentMethod*

paymentStatus' = *paymentStatus*

paymentDate' = *paymentDate*

4.3.4 Error Schema Scenario

Schema Name	Success pre-condition	Failure pre-condition	Remark for Failure
NewPayment	$paymentId? \notin paymentId$ $associatedOrderId? \notin associatedOrderId$	$paymentId? \in paymentId$ $associatedOrderId? \in associatedOrderId$	-Payment exist -Order exist
ViewPaymentDetials	$paymentID? \in paymentID$	$paymentID? \notin paymentID$	-Payment not exist
UpdatePaymentMethod	$paymentID? \in paymentID$ $paymentMethod \neq paymentMethod$ $paymentStatus \neq successful$ $paymentStatus \neq cancelled$	$paymentID? \notin paymentID$ $paymentMethod = paymentMethod$ $paymentStatus = successful$ $paymentStatus = cancelled$	-Payment not exist -New payment method is same with current payment method -Payment already success or cancelled
RemovePayment	$paymentID? \in paymentID$ $paymentStatus \neq successful$	$paymentID? \notin paymentID$ $paymentStatus = successful$	-Payment not exist -Payment is already successful
UpdatePaymentStatus	$paymentID? \in paymentID$ $paymentStatus = paymentStatus?$ $paymentStatus \neq successful$ $paymentStatus \neq cancelled$	$paymentID? \notin paymentID$ $paymentStatus = paymentStatus?$ $paymentStatus = successful$ $paymentStatus = cancelled$	-Payment not exist -Payment status is same -Payment is already successful or cancelled
RetrievePaymentDetailsUsingOrderId	$associatedOrderId? \in ran(associatedOrderId \triangleright associatedOrderId?)$	$associatedOrderId? \notin ran(associatedOrderId \triangleright associatedOrderId?)$	-Order not exist

PAYMENTRESPOND ::= success | payment_exist| order_exist| payment_not_exist| payment_method_is_same| payment_already_successful| payment_already_cancelled| payment_status_is_same| order_no_found

Success

respond! : PAYMENTRESPOND

respond! = success

NewPaymentError

$\exists$ Payment

paymentId? : PAYMENTID

associatedOrderId? : ORDERID

respond! : PAYMENTRESPOND

$(paymentId? \in paymentId \wedge respond! = payment_exist)$

$\vee$

$(associatedOrderId? \in associatedOrderId \wedge respond! = order_exist)$

ViewPaymentDetailsError

$\exists$ Payment

paymentId? : PAYMENTID

respond! : PAYMENTRESPOND

$(paymentId? \notin paymentId \wedge respond! = payment_not_exist)$

UpdatePaymentMethodError

Δ Payment

paymentID? : PAYMENTID

paymentMethod? : PAYMENTMETHOD

respond! : PAYMENTRESPOND

$(paymentId? \notin paymentId \wedge respond! = payment_not_exist)$

$\vee$

$(paymentMethod = paymentMethod? \wedge respond! = payment_method_is_same)$

$\vee$

$(paymentStatus = successful \wedge respond! = payment_already_successful)$

$\vee$

$(paymentStatus = cancelled \wedge respond! = payment_already_cancelled)$

RemovePaymentError

Δ Payment

$paymentId? : PAYMENTID$

$respond! : PAYMENTRESPOND$

$(paymentId? \notin paymentId \wedge respond! = payment_not_exist)$

$\vee$

$(paymentStatus = successful \wedge respond! = payment_already_successful)$

UpdatePaymentStatusError

Δ Payment

$paymentId? : PAYMENTID$

$paymentStatus? = PAYMENTSTATUS$

$respond! : PAYMENTRESPOND$

$(paymentId? \notin paymentId \wedge respond! = payment_not_exist)$

$\vee$

$(paymentStatus = paymentStatus? \wedge respond! = payment_status_is_same)$

$\vee$

$(paymentStatus = successful \wedge respond! = payment_already_successful)$

$\vee$

$(paymentStatus = cancelled \wedge respond! = payment_already_cancelled)$

RetrievePaymentDetailsUsingOrderIdError

Ξ Payment

$associatedOrderId? = ORDERID$

$respond! : PAYMENTRESPOND$

$(associatedOrderId? \notin ran(associatedOrderId \triangleright associatedOrderId?) \wedge respond! = order_no_found)$

4.3.5 Complete Schema

- 1) $NewPaymentComplete \triangleq (NewPayment \wedge Success) \vee NewPaymentError$
- 2) $ViewPaymentDetailsComplete \triangleq (ViewPaymentDetails \wedge Success) \vee ViewPaymentDetailsError$
- 3) $UpdatePaymentMethodComplete \triangleq (UpdatePaymentMethod \wedge Success) \vee UpdatePaymentMethodError$
- 4) $RemovePaymentComplete \triangleq (RemovePayment \wedge Success) \vee RemovePaymentError$
- 5) $UpdatePaymentStatusComplete \triangleq (UpdatePaymentStatus \wedge Success) \vee UpdatePaymentStatusError$
- 6) $RetrievePaymentDetailsUsingOrderIdComplete \triangleq (RetrievePaymentDetailsUsingOrderId \wedge Success) \vee RetrievePaymentDetailsUsingOrderIdError$

4.4 Membership (Dylan Kong Jun Hong)

4.4.1 State Space Schema

Membership

membershipID : $\mathbb{P}$ *MEMBERSHIPID*

membershipLevel : $\mathbb{P}$ (*MEMBERSHIPID* $\times$ *MEMBERSHIPLEVEL*)

membershipStatus : $\mathbb{P}$ (*MEMBERSHIPID* $\times$ *MEMBERSHIPSTATUS*)

point : $\mathbb{P}$ (*MEMBERSHIPID* $\times$ *POINT*)

convertRate : $\mathbb{P}$ (*MEMBERSHIPID* $\times$ *CONVERTRATE*)

historyExchange : $\mathbb{P}$ (*MEMBERSHIPID* $\times$ *HISTORYEXCHANGE*)

reward : $\mathbb{P}$ (*MEMBERSHIPID* $\times$ *REWARD*)

expiryDate : $\mathbb{P}$ (*MEMBERSHIPID* $\times$ *EXPIRYDATE*)

dom membershipLevel = *dom membershipStatus* = *dom point* = *dom convertRate* =

dom history = *dom reward* = *dom expiryDate* = *membershipId*

CustomerPoint

membershipID : $\mathbb{P}$ *MEMBERSHIPID*

membershipLevel : $\mathbb{P}$ (*MEMBERSHIPID* $\times$ *MEMBERSHIPLEVEL*)

membershipStatus : $\mathbb{P}$ (*MEMBERSHIPID* $\times$ *MEMBERSHIPSTATUS*)

point : $\mathbb{P}$ (*MEMBERSHIPID* $\times$ *POINT*)

convertRate : $\mathbb{P}$ (*MEMBERSHIPID* $\times$ *CONVERTRATE*)

reward : $\mathbb{P}$ (*MEMBERSHIPID* $\times$ *REWARD*)

expiryDate : $\mathbb{P}$ (*MEMBERSHIPID* $\times$ *EXPIRYDATE*)

membershipStatus $\subseteq$ *membershipID*

point $\subseteq$ *membershipID*, *membershipLevel*

Redeemable

membershipID : $\mathbb{P}$ *MEMBERSHIPID*

membershipLevel : $\mathbb{P}$ (*MEMBERSHIPID* $\times$ *MEMBERSHIPLEVEL*)

membershipStatus : $\mathbb{P}$ (*MEMBERSHIPID* $\times$ *MEMBERSHIPSTATUS*)

point : $\mathbb{P}$ (*MEMBERSHIPID* $\times$ *POINT*)

reward : $\mathbb{P}$ (*MEMBERSHIPID* $\times$ *REWARD*)

dom membershipID = *dom membershipLevel* = *dom membershipStatus* = *dom point* = *dom reward*

4.4.2 Initial State Space Scheme

InitMembership

Membership

$membershipID = \emptyset$
 $membershipLevel = \emptyset$
 $membershipStatus = \emptyset$
 $point = \emptyset$
 $reward = \emptyset$
 $expiryDate = \emptyset$
 $historyExchange = \emptyset$
 $convertRate = \emptyset \rightarrow$ the membership initiating from null value.

InitCustomerPoints

CustomerPoint

$membershipID = \emptyset$
 $membershipLevel = \emptyset$
 $membershipStatus = \emptyset$
 $point = \emptyset$
 $reward = \emptyset$
 $convertRate = \emptyset$
 $historyExchange = \emptyset$
 $expiryDate = \emptyset$

InitRedeemable

Redeemable

$point \geq reward$
 $reward = \emptyset$
 $historyExchange = \emptyset$
 $expiryDate = \emptyset$

4.4.3 Operation Schema

4.4.3.1 Operation 1: CRUD - Point Module

AddPoint

Δ Membership

newMSID? : *MEMBERSHIPID*

newCust? : *CUSTOMERID*

initialLevel? : *MEMBERSHIPLEVEL*

initialStatus? : *MEMBERSHIPSTATUS*

initialPoints? : $\mathbb{N}$

expiryDate? : *DATE*

$newMSID = newMSID \notin membershipID$

$membership' = membership \cup \{newMSID? \mapsto newCust?\}$

$membershipLevel' = level \cup \{newMSID \mapsto initialLevel\}$

$membershipStatus' = membershipStatus \cup \{newMSID \mapsto initialStatus?\}$

$point' = point \cup \{newMSID \mapsto initialPoints?\}$

$reward' = reward$

$historyExchange' = historyExchange$

DeletePoint

Δ Membership

deleteID? : *MEMBERSHIP*

$newMSID \notin membershipID$

$membershipLevel' = membershipLevel' \setminus \{deleteID?\}$

$membershipStatus' = membershipStatus' \setminus \{deleteID?\}$

$point' = point \setminus \{deleteID?\}$

$convertRate' =$

$historyExchange' = historyExchange \setminus \{deleteID?\}$

$reward' = reward \setminus \{deleteID?\}$

$expiryDate' = expiryDate \setminus \{deleteID?\}$

ViewPointHistoryDetails

$\exists$ Membership

$queryMSID? : MEMEBERSHIPID$

$viewPointHistoryDetails! = \mathbb{P} (MEMBERSHIP \times HISTORYEXCHANGE)$

$newMSID \notin membershipID$

$level' = membershipLevel$

$status' = membershipStatus$

$point' = point$

$convertRate' = convertRate$

$historyExchange' = historyExchange$

$reward' = reward$

$expiryDate' = expiryDate$

$viewPointHistoryDetails! = (membership\ queryID?, membershipLevel\ queryID?, membershipStatus\ queryID?, point\ queryID?, reward\ queryID?)$

UpdatePoints

$\Delta Membership$

$updateID? : MEMBERSHIPID$

$newLevel? : MEMBERSHIPLEVEL$

$addPoint? : \mathbb{N}$

$newMSID \notin membershipID$

$point' = updateID? + addPoint?$

$level' = level' \cup \{updateID? \mapsto newLevel/\}$

$point' = point' \cup \{updateID? \mapsto point\ updateID? + addPoint\}$

$convertRate' = converRate$

$historyExchange' = historyExchange \cup \{historyExchange?\}$

$reward' = reward \cup \{addPoint?\}$

$expiryDate' = expiryDate$

4.4.3.2 Operation 2: Redeem Module

RedeemReward

Δ *Membership*

redeemID? : *MEMBERSHIPID*

reward? : *REDEEMABLE*

redeemID? $\in \text{dom } \textit{reward}$

reward? $\in \textit{reward } \textit{redeemID?}$

$\textit{reward}' = \textit{reward} \cup \{\textit{redeemID?} \mapsto \textit{redeemID?}\} \setminus \{\textit{reward?}\}$

$\textit{historyExchange}' = \textit{historyExchange} \cup \{(\textit{redeemID?}, \textit{reward?})\}$

4.4.3.3 Operation 3: Change Point

changePoints

Membership

memberID? : *MEMBERSHIPID*

newPoints? : $\mathbb{N}$

memberID? $\in \text{dom } \textit{point}$

$\textit{point}' = \textit{point} \cup \{\textit{memberID?} \mapsto \textit{newPoint?}\}$

4.4.3.4 Operation 4: Deal Module

AddDeal

MemberExclusiveDeals

newDealID? : *DEALID*

newTitle? : *TITLE*

newDealType? : *DEALTYPE*

requiredLevel? : *MEMBERSHIPLEVEL*

expiryDate? : *DATE*

result! : *BOOL*

newDealID? $\notin \text{dom } \textit{deals}$

$\textit{deals}' = \textit{deals} \cup \{ \textit{newDealID?} \mapsto (\textit{newTitle?}, \textit{newDealType?}, \textit{requiredLevel?}, \textit{requiredStatus?}, \textit{expiryDate?}, \textit{true}) \}$

$\textit{redeemed}' = \textit{redeemed}$

$\textit{result!} = \textit{true}$

EligibleDeals

$\exists \textit{MemberExclusiveDeals}$

memberID? : *MEMBERSHIPID*

memberLevel? : *MEMBERLEVEL*

memberStatus? : *MEMBERSHIPSTATUS*

eligibleDeals! : $\mathbb{P} \textit{DEAL}$

$\textit{memberID} \in \text{dom } \textit{membership}$

$\wedge \textit{membershipStatus}(\textit{memberID?}) = \textit{member}$

$\textit{eligibleDeals!} = \{ d : \textit{DEALID} \mid d \in \textit{deals} \wedge$

$\textit{eligibility}(d) \subseteq \{ \textit{bronze}, \textit{silver}, \textit{gold} \} \wedge \textit{membershipLevel}(\textit{memberID?}) \in \textit{eligibility}(d) \}$

outputDeals! : The set of deals the member is eligible for

DeactivateExpiredDeals

$\Delta \textit{MemberExclusiveDeals}$

today : *EXPIRYDATE*

$\textit{expiryDates} = \{ d \in \text{dom } \textit{deals} \bullet \textit{expiryDates}(d) \}$

$\textit{activeDeals} = \{ d \in \text{dom } \textit{deals} \bullet \textit{dealStatus}(d) = \textit{active} \}$

$\textit{expiryDates} \subseteq \textit{activeDeals} \vee \textit{expiryDates} \notin \textit{today}$

$deals' = \{(expiryDates - expiredDeals) \mid activeDeals\}$

outputDeals! : The set of deals that the member is eligible to redeem.

RedeemableDeal

$\exists MemberExclusiveDeals$

$memberID? : MEMBERSHIPID$

$outputDeals! : \mathbb{P} DEAL$

$memberID? \in \text{dom } membership$

$\wedge membershipStatus(memberID?) = member$

$outputDeals! = \{d : DEAL \mid d \in \text{dom } deals \wedge redeemable(d) = true \wedge membershipLevel(memberID?) \in eligibility(d)\}$

4.4.4 Error Scenario Schema

AddMembership

Membership

newID? : MEMBERSHIPID
newCust? : CUSTOMERID
initialLevel? : MEMBERSHIPLEVEL
initialStatus? : MEMBERSHIPSTATUS
initialPoint? : $\mathbb{N}$
expiryDate : DATE
result! : BOOL
error! : $\mathbb{P}$ STRING

newID? $\notin$ *membership* // *membership* already exists

result! = true $\wedge$ *error!* = $\emptyset$
membership' = *membership* $\cup$ {*newID?* $\mapsto$ *newCustomer?*} //
level' = *level* $\cup$ {*newID?* $\mapsto$ *initialLevel?*}
status' = *status* $\cup$ {*newID?* $\mapsto$ *initialLevel?*}
expiryDate' = *expiryDate* $\cup$ {*newID?* $\mapsto$ *expiryDate?*}
reward' = *reward*
historyExchange' = *historyExchange*

result! = false $\Rightarrow$ *error!* =
newID? $\in$ dom *membership* $\wedge$
errorMessage! = “Membership ID ”

ErrorState

errorMessage: $\mathbb{P}$ STRING

errorMessage $\in$ {“Membership ID already exists”,
“Membership not found”,
“Invalid membership level”,
“Invalid date”}

DeleteMembership

Δ Membership

deleteID?: MEMBERSHIPID
result!: BOOL

error!: $\mathbb{P}$ *STRING*

pre-conditions

deleteID? $\in$ *dom membership*

status(deleteID?) $\neq$ *member*

point(deleteID?) $= 0 \wedge \forall r \in \text{reward}(\text{deleteID?}) \bullet r.\text{active} = \text{false}$

expiryDate(deleteID?) $\leq$ *today*

$\forall h \in \text{history}(\text{deleteID?}) \bullet h.\text{unresolved} = \text{false}$

deleteID? $\notin$ *dom membership* $\vee$

$(\text{deleteID?} \in \text{dom membership} \wedge \text{status}(\text{deleteID?}) = \text{member}) \vee$

$(\text{deleteID?} \in \text{dom membership} \wedge \text{point}(\text{deleteID?}) > 0) \vee$

$(\text{deleteID?} \in \text{dom membership} \wedge \text{expiryDate}(\text{deleteID?}) > \text{today}) \vee$

$(\text{deleteID?} \in \text{dom membership} \wedge \exists h \in \text{history}(\text{deleteID?}) \bullet h.\text{unresolved} = \text{true})$

4.4.5 Complete Schema

CustomerPoint&Redeemable

CustomerPointAndRedemable

membershipID : $\mathbb{P}$ *MEMBERSHIPID*

membershipLevel : *MEMBERSHIPID* $\leftrightarrow$ *MEMBERSHIPLEVEL*

membershipStatus : *MEMBERSHIPID* $\leftrightarrow$ *MEMBERSHIPSTATUS*

point : *MEMBERSHIPID* $\leftrightarrow$ *POINT*

convertRate : *MEMBERSHIPID* $\leftrightarrow$ *CONVERTRATE*

reward : *MEMBERSHIPID* $\leftrightarrow$ *REWARD*

expiryDate : *MEMBERSHIPID* $\leftrightarrow$ *EXPIRYDATE*

membershipLevel $\subseteq$ *membershipID* $\times$ *MEMBERSHIPLEVEL*

membershipStatus $\subseteq$ *membershipID* $\times$ *MEMBERSHIPSTATUS*

point $\subseteq$ *membershipID* $\times$ *POINT*

convertRate $\subseteq$ *membershipID* $\times$ *CONVERTRATE*

reward $\subseteq$ *membershipID* $\times$ *REWARD*

expiryDate $\subseteq$ *membershipID* $\times$ *EXPIRYDATE*

dom membershipID = *dom membershipLevel* = *dom membershipStatus* = *dom point* = *dom reward*

MemberExclusiveDeals&Redeemable

MemberExclusiveDealsAndRedeemable

$membershipID : \mathbb{P} \text{ MEMBERSHIPID}$

$membershipLevel : \text{MEMBERSHIPID} \leftrightarrow \text{MEMBERSHIPLEVEL}$

$membershipStatus : \text{MEMBERSHIPID} \leftrightarrow \text{MEMBERSHIPSTATUS}$

$point : \text{MEMBERSHIPID} \leftrightarrow \text{POINT}$

$reward : \text{MEMBERSHIPID} \leftrightarrow \text{REWARD}$

$dealID: \mathbb{P} \text{ DEALID}$

$membershipStatus : \text{DEALID} \leftrightarrow \text{MEMBERSHIPSTATUS}$

$point : \text{DEALID} \leftrightarrow \text{POINT}$

$dealType : \text{DEALID} \leftrightarrow \text{DEALTYPE}$

$redeemed: \text{MEMBERSHIPID} \leftrightarrow \text{DEALID}$

$membershipLevel \subseteq membershipID \times \text{MEMBERSHIPLEVEL}$

$membershipStatus = membershipID \times \text{MEMBERSHIPSTATUS}$

$point \subseteq membershipID \times \text{POINT}$

$reward \subseteq membershipID \times \text{REWARD}$

$\text{dom } redeemed \subseteq membershipID$

$\text{ran } redeemed \subseteq dealID$

4.5 Customer Service & Support (Tan Siew Wei)

4.5.1 State Space Schema

CustomerInquiry

$custID : \mathbb{P} CUSTOMERID$
 $staffID : \mathbb{P} STAFFID$
 $enquiryDate : CUSTOMERID \rightarrow DATE$
 $enquiryStatus : CUSTOMERID \rightarrow ENQUIRYSTATUS$
 $enquiryPriority : CUSTOMERID \rightarrow ENQUIRYPRIORITY$
 $enquiryText : CUSTOMERID \rightarrow ENQUIRYTEXT$
 $feedbackText : CUSTOMERID \rightarrow FEEDBACKTEXT$
 $inquiryAssignedStaff : CUSTOMERID \rightarrow STAFFID$

$dom\ enquiryDate = dom\ enquiryStatus = dom\ enquiryPriority = dom\ enquiryText = dom\ feedbackText =$
 $dom\ inquiryAssignedStaff = custID$
 $ran\ inquiryAssignedStaff = staffID$

OrderRefund

$custID : \mathbb{P} CUSTOMERID$
 $staffID : \mathbb{P} STAFFID$
 $orderRefundRequest : CUSTOMERID \rightarrow BOOLEAN$
 $orderRefundStatus : CUSTOMERID \rightarrow ORDERREFUNDSTATUS$
 $orderRefundText : CUSTOMERID \rightarrow ORDERREFUNDTEXT$
 $refundAssignedStaff : CUSTOMERID \rightarrow STAFFID$

$dom\ orderRefundRequest = dom\ orderRefundStatus = dom\ orderRefundText = dom\ refundAssignedStaff = custID$
 $ran\ refundAssignedStaff = staffID$

StaffResponse

$staffID : \mathbb{P} STAFFID$
 $custID : \mathbb{P} CUSTOMERID$
 $assignedInquiry : STAFFID \rightarrow CUSTOMERID$
 $assignedRefund : STAFFID \rightarrow CUSTOMERID$
 $followUpDate : STAFFID \rightarrow FOLLOWUPDATE$
 $staffResponse : STAFFID \rightarrow STAFFRESPONSE$

$dom\ assignedInquiry = dom\ assignedRefund = dom\ followUpDate = dom\ staffResponse = staffID$
 $ran\ assignedInquiry = ran\ assignedRefund = custID$

CustomerServiceAndSupport

CustomerInquiry
OrderRefund
StaffResponse

$CustomerInquiry \cup OrderRefund \cup StaffResponse$

4.5.2 Initial State Space Scheme

<i>InitCustomerInquiry</i>	
<i>CustomerInquiry</i>	
<i>staffID</i> = $\emptyset$	
<i>custID</i> = $\emptyset$	

<i>InitOrderRefund</i>	
<i>OrderRefund</i>	
<i>staffID</i> = $\emptyset$	
<i>custID</i> = $\emptyset$	

<i>InitStaffResponse</i>	
<i>StaffResponse</i>	
<i>staffID</i> = $\emptyset$	
<i>custID</i> = $\emptyset$	

4.5.3 Operation Schema

Operation 1 : CRUD - Manage Customer Enquiries

AddNewInquiry

Δ CustomerInquiry

Δ StaffResponse

newCustomer? : CUSTOMERID

assignToStaff? : STAFFID

eDate? : DATE

eTXT? : ENQUIRYTEXT

detectedPriority? : ENQUIRYPRIORITY

newCustomer? $\notin$ custID

assignToStaff? $\in$ staffID

detectedPriority? $\in$ {none, low, high}

custID' = *custID* $\cup$ { *newCustomer?* }

enquiryDate' = *enquiryDate* $\cup$ { *newCustomer?* $\mapsto$ *eDate?* }

enquiryStatus' = *enquiryStatus* $\cup$ { *newCustomer?* $\mapsto$ pending }

enquiryPriority' = *enquiryPriority* $\cup$ { *newCustomer?* $\mapsto$ *detectedPriority?* }

enquiryText' = *enquiryText* $\cup$ { *newCustomer?* $\mapsto$ *eTXT?* }

feedbackText' = *feedbackText*

inquiryAssignedStaff' = *inquiryAssignedStaff* $\cup$ { *newCustomer?* $\mapsto$ *assignToStaff?* }

staffID' = *staffID*

assignedInquiry' = *assignedInquiry* $\cup$ { *assignToStaff?* $\mapsto$ *newCustomer?* }

assignedRefund' = *assignedRefund*

followUpDate' = *followUpDate*

staffResponse' = *staffResponse*

UpdateInquiry

Δ CustomerInquiry

Δ StaffResponse

currentCust? : CUSTOMERID

currentStaff? : STAFFID

eText! : ENQUIRYTEXT

upDated? : FOLLOWUPDATE

response? : STAFFRESPONSE

currentCust? $\in$ *custID*

currentStaff? $\in$ *staffID*

eText! $\neq \emptyset$

$(\text{currentCust?} \mapsto \text{currentStaff?}) \in \text{assignedInquiry}$

custID' = *custID*

enquiryDate' = *enquiryDate*

enquiryStatus' = *enquiryStatus* $\oplus$ { *currentCust?* $\mapsto$ resolved }

enquiryPriority' = *enquiryPriority* $\oplus$ { *currentCust?* $\mapsto$ none }

enquiryText' = *enquiryText*

feedbackText' = *feedbackText*

inquiryAssignedStaff' = *inquiryAssignedStaff*

staffID' = *staffID*

assignedInquiry' = *assignedInquiry*

assignedRefund' = *assignedRefund*

followUpDate' = *followUpDate* $\oplus$ { *currentStaff?* $\mapsto$ upDated? }

staffResponse' = *staffResponse* $\oplus$ { *currentStaff?* $\mapsto$ response? }

RemoveCustomerInquiry

Δ CustomerInquiry

Δ StaffResponse

staffRemoveAction? : STAFFID

removeCustInquiry? : CUSTOMERID

currentDate? : DATE

$(\text{removeCustInquiry?} \mapsto \text{resolved}) \in \text{custID}$

$((\text{currentDate?} - \text{enquiryDate}(\text{removeCustInquiry?})) > \text{inquiryMonthLimit}) \in \text{DATE}$

$\text{custID}' = \text{custID}$

$\text{enquiryDate}' = \text{enquiryDate}$

$\text{enquiryStatus}' = \text{enquiryStatus} \setminus \{ \text{removeCustInquiry?} \mapsto \text{resolved} \}$

$\text{enquiryPriority}' = \text{enquiryPriority} \setminus \{ \text{removeCustInquiry?} \}$

$\text{enquiryText}' = \text{enquiryText} \setminus \{ \text{removeCustInquiry?} \}$

$\text{feedbackText}' = \text{feedbackText} \setminus \{ \text{removeCustInquiry?} \}$

$\text{inquiryAssignedStaff}' = \text{inquiryAssignedStaff} \setminus \{ \text{removeCustInquiry?} \}$

$\text{staffID}' = \text{staffID}$

$\text{assignedInquiry}' = \text{assignedInquiry} \setminus \{ \text{staffRemoveAction?} \}$

$\text{assignedRefund}' = \text{assignedRefund}$

$\text{followUpDate}' = \text{followUpDate}$

$\text{staffResponse}' = \text{staffResponse} \setminus \{ \text{staffRemoveAction?} \}$

ViewImportantInquiry

⊖ CustomerInquiry

⊖ StaffResponse

$s? : ENQUIRYSTATUS$

$p? : ENQUIRYPRIORITY$

$startDate? : DATE$

$endDate? : DATE$

$viewInquiries! : \mathbb{P} (CUSTOMERID \times ENQUIRYSTATUS \times ENQUIRYPRIORITY \times DATE \times INQUIRYTEXT \times STAFFID)$

$totalInquiry! : \mathbb{N}$

$s? \in \{pending, resolved, reviewed\}$

$p? \in \{none, low, high\}$

$startDate? \leq endDate?$

$viewInquiries! = \{cID : CUSTOMERID; stu : ENQUIRYSTATUS; pri : ENQUIRYPRIORITY;$

$d : DATE; i : INQUIRYTEXT; sID : STAFFID \mid$

$((cID \mapsto stu) \in enquiryStatus \wedge stu = s?) \wedge ((cID \mapsto pri) \in enquiryPriority \wedge pri = p?) \wedge$

$(cID \mapsto d) \in enquiryDate \wedge (cID \mapsto i) \in enquiryText \wedge (cID \mapsto sID) \in refundAssignedStaff \wedge$

$(startDate? \leq d \leq endDate?)\}$

$totalInquiry! = \# (\{cID : CUSTOMERID \mid (cID \mapsto p?) \in enquiryPriority\})$

$custID' = custID$

$enquiryDate' = enquiryDate$

$enquiryStatus' = enquiryStatus$

$enquiryPriority' = enquiryPriority$

$enquiryText' = enquiryText$

$feedbackText' = feedbackText$

$inquiryAssignedStaff' = inquiryAssignedStaff$

$staffID' = staffID$

$assignedInquiry' = assignedInquiry$

$assignedRefund' = assignedRefund$

$followUpDate' = followUpDate$

$staffResponse' = staffResponse$

Operation 2 : Manage Order Request

ManageOrderRefundRequest

Δ OrderRefund

Δ StaffResponse

$cID? : CUSTOMERID$

$sID? : STAFFID$

$rStatus? : ODERREFUNDSTATUS$

$rText? : ORDERREFUNDTEXT$

$result! : \mathbb{P} \{success\}$

$cID? \in dom\ orderRefundRequest$

$sID? \in staffID$

$rStatus? \in \{requested, approved, rejected\}$

$custID' = custID$

$orderRefundText' = orderRefundText$

$rStatus? = requested \Rightarrow (orderRefundRequest(cID?) = true \wedge$

$orderRefundStatus' = orderRefundStatus \oplus \{cID? \mapsto requested\} \wedge$

$refundAssignedStaff' = refundAssignedStaff \oplus \{cID? \mapsto sID?\} \wedge$

$result! = success)$

$rStatus? = approved \Rightarrow (orderRefundRequest(cID?) = true \wedge$

$orderRefundStatus' = orderRefundStatus \oplus \{cID? \mapsto approved\} \wedge$

$refundAssignedStaff' = refundAssignedStaff \oplus \{cID? \mapsto sID?\} \wedge$

$result! = success)$

$rStatus? = rejected \Rightarrow (orderRefundRequest(cID?) = false \wedge$

$orderRefundStatus' = orderRefundStatus \oplus \{cID? \mapsto rejected\} \wedge$

$refundAssignedStaff' = refundAssignedStaff \oplus \{cID? \mapsto sID?\} \wedge$

$result! = success)$

$staffID' = staffID$

$assignedInquiry' = assignedInquiry$

$assignedRefund' = assignedRefund$

$followUpDate' = followUpDate$

$staffResponse' = staffResponse$

Operation 3 : Customer Feedback

CustomerFeedback

Δ *CustomerInquiry*

Δ *StaffResponse*

cID? : *CUSTOMERID*

sID? : *STAFFID*

feedbackText? : *FEEDBACKTEXT*

responseText? : *STAFFRESPONSE*

cID? $\in$ *custID*

sID? $\in$ *staffID*

feedbackText? $\neq \emptyset$

enquiryStatus' = *enquiryStatus* $\oplus$ {*cID?* $\mapsto$ *reviewed*}

feedbackText' = *feedbackText* $\oplus$ {*cID?* $\mapsto$ *feedbackText?*}

inquiryAssignedStaff' = *inquiryAssignedStaff* $\oplus$ {*cID?* $\mapsto$ *sID?*}

staffResponse' = *staffResponse* $\oplus$ {*sID?* $\mapsto$ *responseText?*}

custID' = *custID*

enquiryDate' = *enquiryDate*

enquiryPriority' = *enquiryPriority*

enquiryText' = *enquiryText*

staffID' = *staffID*

assignedInquiry' = *assignedInquiry*

assignedRefund' = *assignedRefund*

followUpDate' = *followUpDate*

4.5.4 Error Schema Scenario

Schema Name	Success pre-condition	Failure pre-condition	Remark for Failure
Add New Inquiry	$newCustomer? \notin custID$ $assignToStaff? \in staffID$ $detectedPriority? \in \{none, low, high\}$	$newCustomer? \in custID$ $assignToStaff? \notin staffID$ $detectedPriority? \notin \{none, low, high\}$	Customer exist Staff not exist Detect priority not match
Update Inquiry	$currentCust? \in custID$ $currentStaff? \in staffID$ $eText! \neq \emptyset$ $(currentCust? \mapsto currentStaff?) \in assignedInquiry$	$currentCust? \notin custID$ $currentStaff? \notin staffID$ $eText! = \emptyset$ $(currentCust? \mapsto currentStaff?) \notin assignedInquiry$	Customer not exist Staff not exist Enquiry Text not empty Inquiry not assigned
Remove Customer Inquiry	$(removeCustInquiry? \mapsto resolved) \in custID$ $((currentDate? - enquiryDate(removeCustInquiry?)) > inquiryMonthLimit \in DATE$	$(removeCustInquiry? \mapsto resolved) \notin custID$ $((currentDate? - enquiryDate(removeCustInquiry?)) > inquiryMonthLimit \notin DATE$	Customer not exist Inquiry not older than 6 month
View Important Inquiry	$s? \in \{pending, resolved, reviewed\}$ $p? \in \{none, low, high\}$ $startDate? \leq endDate?$	$s? \notin \{pending, resolved, reviewed\}$ $p? \notin \{none, low, high\}$ $startDate? \geq endDate?$	Status not match Priority not match Invalid date
Manage Order Refund Request	$cID? \in dom\ orderRefundRequest$ $sID? \in staffID$ $rStatus? \in \{requested, approved, rejected\}$	$cID? \notin dom\ orderRefundRequest$ $sID? \notin staffID$ $rStatus? \notin \{requested, approved, rejected\}$	Refund not exist Staff not exist Status not match
Customer Feedback	$cID? \in custID$ $sID? \in staffID$ $feedbackText? \neq \emptyset$	$cID? \notin custID$ $sID? \notin staffID$ $feedbackText? = \emptyset$	Customer not exist Staff not exist Feedback Text cannot empty

RESPONSE ::= success | customer_exist | staff_not_exist | customer_not_exist | detectPriority_not_match | enquiryText_not_empty | inquiry_not_assignedByStaff | inquiry_not_olderThan_6_month | status_not_match | priority_not_match | invalid_date | refund_not_exist | feedbackText_not_empty

Success

rep! : RESPONSE

rep! = success

AddNewInquiryError

$\exists$ CustomerInquiry

$\exists$ StaffResponse

newCustomer? : CUSTOMERID

assignToStaff? : STAFFID

detectedPriority? : ENQUIRYPRIORITY

rep! : RESPONSE

(newCustomer? $\in$ custID $\wedge$ rep! = customer_exist)

$\vee$

(assignToStaff? $\notin$ staffID $\wedge$ rep! = staff_not_exist)

$\vee$

(detectedPriority? $\notin$ {none, low, high} $\wedge$ rep! = detectPriority_not_match)

UpdateInquiryError

$\exists$ CustomerInquiry

$\exists$ StaffResponse

currentCust? :CUSTOMERID

currentStaff? : STAFFID

eText! : ENQUIRYTEXT

rep! : RESPONSE

(currentCust? $\notin$ custID $\wedge$ rep! = customer_not_exist)

$\vee$

(currentStaff? $\notin$ staffID $\wedge$ rep! = staff_not_exist)

$\vee$

(eText! = $\emptyset$ $\wedge$ rep! = enquiryText_not_empty)

$\vee$

((currentCust? $\mapsto$ currentStaff?) $\notin$ assignedInquiry $\wedge$ rep! = inquiry_not_assigned)

RemoveCustomerInquiryError

$\exists$ CustomerInquiry

$\exists$ StaffResponse

removeCustInquiry? : CUSTOMERID

currentDate? : DATE

rep! : RESPONSE

$((\text{removeCustInquiry?} \mapsto \text{resolved}) \notin \text{custID} \wedge \text{rep!} = \text{customer_not_exist})$

$\vee$

$((\text{currentDate?} - \text{enquiryDate}(\text{removeCustInquiry?}) > \text{inquiryMonthLimit} \notin \text{DATE} \wedge \text{rep!} = \text{inquiry_not_olderThan_6_month})$

ViewImportantInquiryError

$\exists$ CustomerInquiry

$\exists$ StaffResponse

s? : ENQUIRYSTATUS

p? : ENQUIRYPRIORITY

startDate? : DATE

endDate? : DATE

rep! : RESPONSE

$(s? \notin \{\text{pending, resolved, reviewed}\} \wedge \text{rep!} = \text{status_not_found})$

$\vee$

$(p? \notin \{\text{none, low, high}\} \wedge \text{rep!} = \text{priority_not_match})$

$\vee$

$(\text{startDate?} \geq \text{endDate?} \wedge \text{rep!} = \text{invalid_date})$

ManageOrderRefundRequestError

$\exists$ OrderRefund

$\exists$ StaffResponse

cID? : CUSTOMERID

sID? : STAFFID

rStatus? : ODERREFUNDSTATUS

rep! : RESPONSE

$(cID? \notin \text{dom orderRefundRequest} \wedge \text{rep!} = \text{refund_not_exist})$

$\vee$

$(sID? \notin \text{staffID} \wedge \text{rep!} = \text{staff_not_exist})$

$\vee$

$(rStatus? \notin \{\text{requested, approved, rejected}\} \wedge \text{rep!} = \text{status_not_exist})$

CustomerFeedbackError

$\exists$ *OrderRefund*

$\exists$ *StaffResponse*

cID? : *CUSTOMERID*

sID? : *STAFFID*

feedbackText? : *FEEDBACKTEXT*

rep! : *RESPONSE*

$(cID? \notin custID \wedge rep! = customer_not_exist)$

$\vee$

$(sID? \notin staffID \wedge rep! = staff_not_exist)$

$\vee$

$(feedbackText? = \emptyset \wedge rep! = feedbackText_not_empty)$

4.5.5 Complete Schema

- 1) $AddNewInquiryComplete \triangleq (AddNewInquiry \wedge Success) \vee AddNewInquiryError$
- 2) $UpdateInquiryComplete \triangleq (UpdateInquiry \wedge Success) \vee UpdateInquiryError$
- 3) $RemoveCustomerInquiryComplete \triangleq (RemoveCustomerInquiry \wedge Success) \vee RemoveCustomerInquiryError$
- 4) $ViewImportantInquiryComplete \triangleq (ViewImportantInquiry \wedge Success) \vee ViewImportantInquiryError$
- 5) $ManageOrderRefundRequestComplete \triangleq (ManageOrderRefundRequest \wedge Success) \vee$
 $ManageOrderRefundRequestError$
- 6) $CustomerFeedbackComplete \triangleq (CustomerFeedback \wedge Success) \vee CustomerFeedbackError$

5.0 Self-Reflection

Team member: Chan Jing Xing

Through completing this assignment, I've gained a deeper understanding of various concepts in formal methods, schema design, and their practical application in real-world systems.

I've learned how to define operations and represent them formally using schema notation. Creating operation schemas such as AddCustomer, UpdateStaff, and RecoverPassword required attention to detail and a clear understanding of the system's requirements. This process reinforced the importance of defining pre-conditions, post-conditions, and the expected success or failure outcomes for each operation in a structured manner.

Working through these schemas and focusing on correctness helped me appreciate the utility of formal methods in software design. This approach emphasizes precision and helps avoid ambiguity, which is important for complex systems like point-of-sale applications. It also taught me the value of structured thinking when designing and implementing software.

Team member: Dylan Kong Jun Hong

Formal Methods in Software Engineering is a subject primarily focusing on describing the terms and variables required for developing a system. In abstract, the system will have to be as accurate as possible to prevent critical moments, for verification and validation, proving properties and automation and tool support.

In the process of completing the assignment, and learning this course, I have learned not only the importance of discrete mathematics and the logic behind a system to be studied. All the systems developed are based on logic and discrete mathematics especially in a system that is requiring as precise as possible.

Besides of that, the entirely complex description for our system – 99 SpeedMart Mobile Application System. We have included a feature for allowing customers to be part of a membership of 99 SpeedMart. At the same time, gain exclusive deals and vouchers specially for the customers registered. The development of this subsystem has the needness of learning the formal methods and the language to describe the terms needed for developing the system. To begin with, the optional axiomatic definition, state space schema and initialisation schema for the whole subsystem to have its own description. This subject can be difficult, but there are parts that can be done simple. For instance, the error scenarios basically inverse the predicates pre-defined in the state space schema and query state schema. This action is to list out and resolve the errors that might happen in the future, in the name of error detection and prevention.

In a nutshell, none of the critical systems developed in this universe would not require the use of formal methods. In order to create a critically avoidant environment, we will have to ensure formal methods is being well applied.

Team member: Yap Wing Hong

Working on the Order Module for this assignment has been an enriching experience that helped me develop a deeper understanding of system design, formal methods, and real-world problem-solving.

The Order Module required me to define and implement operations such as viewing orders, adjusting item quantities, removing items, canceling orders, and sending order confirmations. This process enhanced my ability to break down complex user requirements into precise, functional components. Each operation had to be carefully designed to ensure seamless order processing and error-free execution.

Managing potential failures, such as invalid order IDs or exceeding item quantity limits, was a critical aspect of this module. By defining clear failure pre-conditions and corresponding error schemas, I learned how to design systems that anticipate and handle errors effectively. This process also reinforced the importance of providing meaningful feedback to users to guide them through resolving issues.

Handling the Order Module has been a transformative experience. It has taught me the value of precision in system design, the importance of error handling, and the need for collaboration between system components. This module has given me practical insights into how real-world systems operate and prepared me to tackle similar challenges in future projects. Through this assignment, I have gained both technical expertise and a deeper appreciation for the intricacies of order management in software systems.

Team member: Tan Siew Wei

My Name is Tan Siew Wei. In the moment finishing our assignment, I have learned that the importance formal methods in software engineering. For instance, defining the axiomatic definition, state space schema and the initialization schema. I am responsible for handling Account Management Subsystem for this assignment. To start this assignment, I have referred to our projects from another subject, which is Software Requirements Engineering to developed this subsystem.

The importance of learning formal methods in software engineering is essential because this will determine the reliability of the systems in the future. As a student under Software Engineering programme, I understand the need to learn discrete mathematics in order to do this assignment, the language and the terms, each makes a significance in designing the system. We have included the 4 elements for each of our module which is CRUD(create, retrieve, update, and delete) functions. Since the start of this assignment, I have done my revision as daily routine and had discovered something: The designing steps are fun and easy if you get it right, as long as there are no calculations but logic to be done. I truly appreciate the time and effort spent by me and our cheerful lecturer for giving patience and kindness in teaching this subject.

Team member: Ng Jien Hao

In this semester, I have learnt formal method in software engineering, Z language is used in this subject. At the early time of the semester, I have learnt how to differentiate basic type, free type and axiomatic type of variables and recognize the sets. It helps me to define and design variables when the assignment starts. After that, I have learnt how to do state space schema. It helps me to define the relations of variables for my system. Other than that, I have learnt operation schema such as CRUD(Create, Retrieve, Update, Delete). The operation schema helps me know change data or retrieve data from my system and set some limitation to prevent errors. Different method of formula can be applied for a situation which can get same results. The next thing I have learnt is relations and functions. It help me to define relations between domain and range of set of data which help me a lot in my operation schema. And the last thing I have learnt to apply in the assignment is error scenario. The operations that against with limitation will be return error message.

6.0 Reference and Appendices

1. Barney, N., & Rosencrance, L. (2024, November 27). What is a software requirements specification (SRS)? Search Software Quality. <https://www.techtarget.com/searchsoftwarequality/definition/software-requirements-specification>
2. Indeed. (n.d.). *99 Speedmart Sdn Bhd job titles*. <https://malaysia.indeed.com/cmp/99-Speedmart-Sdn-Bhd/job-titles>
3. 99 Speedmart. (n.d.). *Teams and awards*. <https://99speedmart.com.my/teams-and-awards/>